

Take-Home: Answer Engine for Engineering Documents

Context

Hardware engineering teams in robotics, aerospace, EV, and similar fields work from a messy mix of documents: patents, design documents full of block diagrams and spec tables, bills of materials, and test logs. When an engineer asks a real question, the answer is rarely in one clean paragraph. It may sit in a table on page 14, in a figure label, in a scanned patent page, or in a spreadsheet row.

Your task is to build an **answer engine** that ingests such a corpus and answers natural-language questions with **grounded, cited answers**. Just as importantly, we want you to **show how good it is** through an evaluation you design and run.

We've kept the scope deliberately focused. We'd rather see a small system that you understand end to end, measure honestly, and can reason about than a broad one held together by black boxes. Please submit within 7 days.

What we provide

- **Corpus** (~10 files):
 - **4 patents (PDF)**. At least one is a scanned image-only PDF. Patents use two-column layouts, numbered claims, and figures whose parts are labelled with reference numerals (e.g. "valve 112" in the text maps to the label 112 in FIG. 3).
 - **3 design documents (PDF/DOCX)**. These contain block diagrams, photos or CAD screenshots, and spec tables (e.g. electrical ratings, dimensions, operating limits).
 - **2 structured files**: a CSV test log and an XLSX bill of materials.
- **Dev set**: ~20 questions, each with a reference answer and the evidence location (document and page). Use it to build and tune your system.
- **Hidden test set**: after you submit, we'll run your system on a separate set of similar questions you haven't seen.

Constraints

1. Models are your choice.

You can use any LLM, VLM, or embedding model, open-weight or hosted (via an API key). Tell us what you used and why. If you use a hosted model, make it configurable so we can plug in our own key.

2. Everything else must be open source and run locally.

Parsing, OCR, layout detection, table extraction, image extraction, chunking, indexing, and retrieval must be done with open-source packages running in your own environment.

- **Not allowed**: hosted document-processing or RAG services, such as LlamaParse, Unstructured's hosted API, AWS Textract, Azure Document Intelligence, Google Document AI, Reducto, or managed vector databases.
- **Allowed**: open-source libraries and the models bundled with them (e.g. Docling, Marker,

PyMuPDF, pdfplumber, Camelot, Tesseract, PaddleOCR, Surya, Table Transformer) and self-hosted storage (e.g. Qdrant, pgvector, LanceDB, DuckDB, SQLite).

- **Grey area, clarified:** you may call a general-purpose LLM/VLM inside your pipeline, for example to describe a figure or to help interpret a table. But the pipeline itself (what gets extracted, how it's structured, and how it links back to the source page) must be yours.

3. Know your tools from the inside.

Using an open-source parser is fine; treating it as a black box is not. For each extraction library you rely on, you should be able to explain:

- how it detects page layout (text blocks, tables, figures, headers)
- how it decides reading order, especially on two-column pages
- how it recovers table structure: rows, columns, header cells
- how it locates and crops images and figures
- when OCR kicks in, and which OCR engine it uses
- where it breaks, ideally with an example from our corpus

We'll spend a good part of the follow-up discussion on this.

4. Easy for us to run.

We should be able to clone your repo and run ingestion, querying, and evaluation with one command each (e.g. make ingest, make ask Q="...", make eval). Pin your dependencies and document any system packages (e.g. Tesseract).

Requirements

1. Ingestion and extraction

Your system should turn the raw files into something that can be searched and cited accurately.

- **Text:** extract text in the correct reading order. Two-column patent pages should not come out interleaved line by line. Scanned pages should go through OCR.
- **Tables:** extract tables with their row and column structure intact, so that "the value in row X, column Y" is still answerable. Flattening a table into a run of words loses exactly the information people ask about.
- **Figures and images:** extract figures, link each one to its caption (e.g. "FIG. 2 is a cross-section of...") and make the figure's content findable. How you do this (OCR of labels, VLM descriptions, captions only) is up to you; explain the tradeoff.
- **Structured files:** ingest the CSV and XLSX so they can be queried precisely, for example with SQL, rather than only embedded as text.
- **Provenance:** every extracted piece should carry its source document and page, so answers can be cited.

2. Retrieval and answering

- Use any retrieval approach you like (keyword, vector, hybrid, page-image based, SQL for structured data, or a mix). Explain briefly why you chose it for this corpus.
- Every answer must cite the **document and page** of its evidence.
- When the corpus does not contain the answer, the system should say **"not found"** instead of guessing. A confident wrong answer is worse than no answer.
- A CLI or a minimal HTTP API is enough. No UI needed.
- Please make the output machine-readable so we can score it automatically, in this shape:

```
{
  "answer": "The rated continuous current is 120 A.",
  "citations": [{ "doc": "motor_controller_design.pdf", "page": 7 }],
  "not_found": false
}
```

Question types

Your system should handle these kinds of questions. The examples are illustrative, not from the actual dev set.

Type	Example
Factual lookup from text	"What material is the housing made of in patent US-XXXX?"
Table lookup	"What is the maximum operating temperature listed in the thermal spec table?"
Figure-grounded	"What component is labelled 204 in FIG. 2?"
Structured data	"Which part number in the BOM has the highest unit cost?" / "How many test runs failed?"
Unanswerable	"What is the warranty period for the controller?" (when no document says)

Evaluation

The evaluation is as important as the system. Build a harness that runs with one command and measures quality **at each stage**, so when an answer is wrong you can tell whether extraction, retrieval, or generation caused it.

Extraction quality

You can't measure extraction against the dev set alone, so create a small gold set yourself: hand-annotate a handful of pages that cover a couple of tables, a figure, and a scanned page. Then report how close your extraction is. Choose metrics you can defend, for example:

- character or word error rate for OCR text
- cell-level accuracy for tables (did each value land in the right row and column?)
- whether each figure was found and linked to the right caption

Retrieval quality

- Recall@k against the gold evidence pages in the dev set: for each question, did the right page appear in the top k results?
- Break results down by question type (text, table, figure, structured), since they usually behave very differently.

Answer quality

- **Correctness:** is the answer right? For numbers, decide on a tolerance and state it. For free-text answers, explain how you judged them (manual review, an LLM grader, or both).
- **Citation accuracy:** do the cited pages actually support the answer?
- **"Not found" behaviour:** on unanswerable questions, how often does the system

correctly decline, and how often does it wrongly decline on answerable ones?

Failure analysis

Include a short table of the failures you found, each with the question, what went wrong, and which stage caused it. We value a clear-eyed failure analysis more than a high score.

Please summarise all results in one table in your README.

Deliverables

1. **Git repository** with code and one-command setup for ingestion, querying, and evaluation.
2. **README** covering:
 - architecture overview (a simple diagram is enough)
 - model and library choices, and why
 - evaluation results table and failure analysis
 - known limitations
 - what you would do next with more time
3. **"How extraction works"** (roughly a page, in the repo): for text, tables, images, and OCR, explain what your chosen library does under the hood, why you picked it over alternatives, and one case where it fails on our corpus and what you did or would do about it.
4. **A short note on AI coding tools.** Using them is allowed and expected; just tell us how you used them.

How we'll assess

- **Evaluation:** is it rigorous, stage-by-stage, and honest about weaknesses?
- **Hidden test set results:** especially tables, figures, and correct "not found" answers.
- **Understanding of your extraction stack:** can you explain how and why it works and fails?
- **Engineering quality:** clear structure, readable code, runs on the first try.
- **Judgment:** sensible choices for this corpus, and clarity on what you chose not to build.

Follow-up discussion

A 60–90 minute session with our team:

1. You walk us through your repo, results, and failure analysis.
2. We go deep on how your parser, table extraction, OCR, and image extraction work internally.
3. We add a new document or question type together and see how your system and evaluation hold up.

Questions

Email us if anything is unclear. If you'd rather not wait, make a reasonable assumption and write it down in your README; that's completely fine.